Tunis Business School
Bachelor in Business Analytics

Tunisia Real Estate Price Prediction

A Data Mining Analysis of the Tunisian Property Market

Noursine Amira
Lina Brahmi
Maycem Azaza

Professor Dr. Aloui Donia
Spring 2025–2026

April 21, 2026

video: https://drive.google.com/file/d/153gb6Nzs-rgioAsSmmjDwT-dQp4DydQc/view?usp=drive_link

Contents

1 Dataset Description **2**
- 1.1 Source and Collection . . . 2
- 1.2 Variables . . . 2
- 1.3 Descriptive Statistics . . . 2

2 Problem Statement **5**
- 2.1 Motivation . . . 5
- 2.2 Analytical Tasks . . . 5

3 Exploratory Data Analysis **5**
- 3.1 Missing Values . . . 5
- 3.2 Price Distribution and Log Transformation . . . 6
- 3.3 Price by Neighbourhood . . . 6
- 3.4 Price by Residence Type . . . 6
- 3.5 Living Area vs. Price . . . 6
- 3.6 Amenity Impact on Median Price . . . 6
- 3.7 Correlation Structure . . . 7

4 Data Preparation **10**

5 Methods **11**
- 5.1 Regression Task . . . 11
  - 5.1.1 KNN Regressor ($k = 10$) . . . 11
  - 5.1.2 Decision Tree Regressor (max depth = 12) . . . 11
  - 5.1.3 Random Forest Regressor (300 trees) . . . 12
- 5.2 Classification Task . . . 13
  - 5.2.1 Gaussian Naive Bayes (baseline) . . . 13
  - 5.2.2 Random Forest Classifier (300 trees) . . . 13

6 Results **14**
- 6.1 Regression . . . 14
  - 6.1.1 Feature Importances . . . 15
- 6.2 Classification . . . 15

7 Live Prediction Demo **17**

8 Discussion and Limitations **18**
- 8.1 Key Findings . . . 18
- 8.2 Limitations . . . 18

9 Conclusion **18**

Dataset Description

Source and Collection

The dataset was scraped from two major Tunisian property portals: **menzili.tn** (Tunisia's leading dedicated property listing site) and **tunisie-annonce.com** (one of the oldest classifieds platforms in Tunisia). Both portals publish listings for apartments, villas, duplexes, studios, and penthouses across all Tunisian governorates. The unified CSV contains **1,400 observations** and **15 variables**. Each row is one property listing published by a seller or agency.

Variables

Descriptive Statistics

Key Observation

The mean price (535,180 TND) is substantially higher than the median (436,000 TND), revealing a **right-skewed distribution**. A small number of luxury properties pull the average upward. This makes the median a more reliable measure of a "typical" property price.

Table 1: Dataset Variables

Variable	Type	Description
price	Numeric	Listing price in TND — target variable
surface_habitable	Numeric	Living area (m^2)
chambres	Numeric	Number of bedrooms
salles_de_bain	Numeric	Number of bathrooms
etage	Numeric	Floor number (0 = ground floor)
pieces_totales	Numeric	Total number of rooms
residence_type	Categorical	appartement / villa / duplex / studio / penthouse
neighborhood	Categorical	Neighbourhood or area in Tunisia
balcon, terrasse, ascenseur, piscine, suite, chauffage_central	Boolean	Amenity indicators (1 = present)
cuisine_type	Categorical	Kitchen quality: equipped / semi / not equipped

Table 2: Key Numeric Variable Summary

Variable	**N**	**Mean**	**Std**	**Min**	**Median**	**Max**
price (TND)	1,400	535,180	360,641	82,000	436,000	2,874,000
surface_habitable (m^2)	1,339	130	79	25	111	900
chambres	1,348	3.0	1.0	1	3	6
salles_de_bain	1,299	2.0	1.0	1	2	5
pieces_totales	1,400	6.0	2.0	3	6	13
etage	1,236	2.0	2.1	0	2	11

Problem Statement

Motivation

The Tunisian real estate market suffers from significant **information asymmetry**: buyers and sellers often have very different knowledge of what a fair price looks like for a given property. A data-driven pricing model helps sellers list at realistic prices, helps buyers identify over- or under-priced listings, and allows agencies to provide automated valuation estimates.

Analytical Tasks

This project pursues two complementary tasks:

1. **Regression** — Predict the exact listing price in TND from property attributes. The output is a continuous number (e.g. 350,000 TND).
2. **Classification** — Assign each property to one of four price tiers for market segmentation. The output is a discrete label (Table 3).

Table 3: Price-Tier Definitions

Tier	Label	Price Range (TND)	Count
1	Low	$< 200{,}000$	133
2	Medium	$200{,}000 \leq p < 500{,}000$	690
3	High	$500{,}000 \leq p < 1{,}000{,}000$	433
4	Luxury	$\geq 1{,}000{,}000$	137

Class Imbalance Note

The **Medium** tier accounts for 49% of all listings. This imbalance means a naive model that always predicts "Medium" would achieve 49% accuracy — the classification models must clearly beat this baseline to be useful.

Exploratory Data Analysis

Missing Values

Not all listing portals enforce complete form-filling. As a result, several numeric fields have missing entries. The chart below shows the rate of missingness for each affected column.

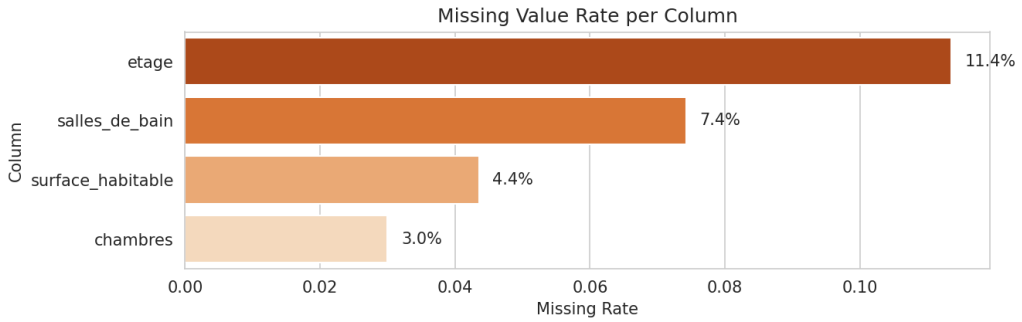


Figure 1: Missing value rate per column. `etage` (floor number) has the highest rate at 11.7%, likely because ground-floor sellers often omit this field. `price` has zero missing values — it is always present on a listing.

`etage` has the highest missingness (11.7%), followed by `salles_de_bain` (7.2%), `surface_habitable` (4.4%), and `chambres` (3.7%). Missingness is considered Missing at Random (MAR) — the probability of a value being absent does not depend on the value itself, only on the property type. All missing values are imputed with column medians (see Section 4).

Price Distribution and Log Transformation

Understanding the shape of the target variable is critical before choosing or evaluating a model. The raw price distribution (left panel below) is heavily right-skewed, with a long tail of luxury properties extending past 2 million TND.



Figure 2: **Left:** Raw price distribution (skewness = 1.88). The long tail toward high prices means most listings cluster at lower values while a few outliers distort the mean. **Right:** Log-transformed price, $\ln(1+p)$ (skewness ≈ 0.15). The distribution is approximately bell-shaped, which satisfies regression model assumptions and reduces the influence of luxury outliers.

Why log-transform the price?

Regression models minimise squared error. If we train on raw prices, a single luxury property at 2.5M TND generates a squared error of $\approx 6 \times 10^{12}$ — dominating the loss and distorting every other prediction. Log-transforming compresses this scale so all properties contribute more equally. Back-transforming via $\hat{p} = e^{\hat{y}} - 1$ recovers the original TND scale for reporting.

Price by Neighbourhood

Location is intuitively one of the strongest price drivers. The figure below confirms this: premium coastal and northern suburbs command dramatically higher prices than inland or southern areas.

Price by Residence Type

Different property types occupy distinct price segments of the market.

Living Area vs. Price

Beyond location and type, how much floor space a property offers is a key value driver.

Amenity Impact on Median Price

Binary amenity features indicate whether a property has specific extras. Their impact on median price reveals which extras buyers value most.

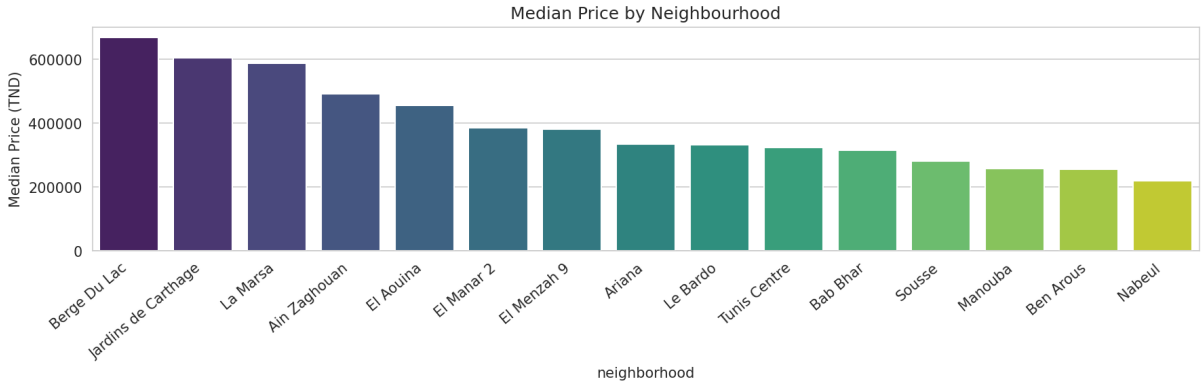


Figure 3: Median listing price by neighbourhood. *Berge du Lac* and *Jardins de Carthage* lead with median prices above 900,000 TND. *Manouba* and *Nabeul* sit at the lower end near 200,000 TND — a more than **4× price gap** across neighbourhoods. This motivates the `neigh_price_index` engineered feature.

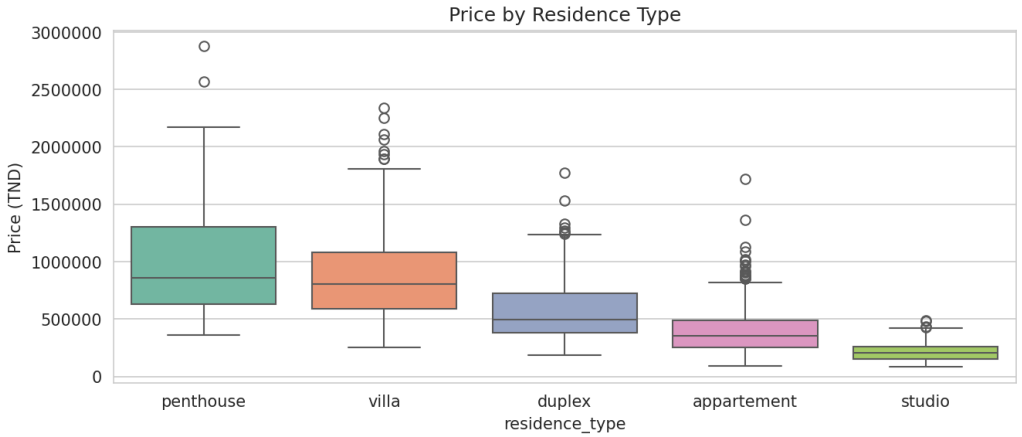


Figure 4: Box plots of price by residence type (ordered by median). Penthouses and villas have both the highest medians and the widest price spreads, indicating heterogeneity within each type. Studios are tightly clustered at the low end with little variation. Apartments and duplexes occupy the broad middle segment.

Correlation Structure

Multicollinearity Warning

The very high correlations among room-count variables ($r > 0.90$) would destabilise a linear regression model's coefficient estimates. Tree-based methods (Decision Tree, Random Forest) are immune to this issue because they select one feature at a time per split, making them better suited to this dataset.



Figure 5: Scatter plot of price against living area, coloured by bedroom count. The positive relationship between surface and price is clear, though with significant scatter around the trend. Properties with more bedrooms (warmer colours) systematically appear at higher prices for a given surface area. The wide vertical scatter at any given surface confirms that size alone is not sufficient to determine price.

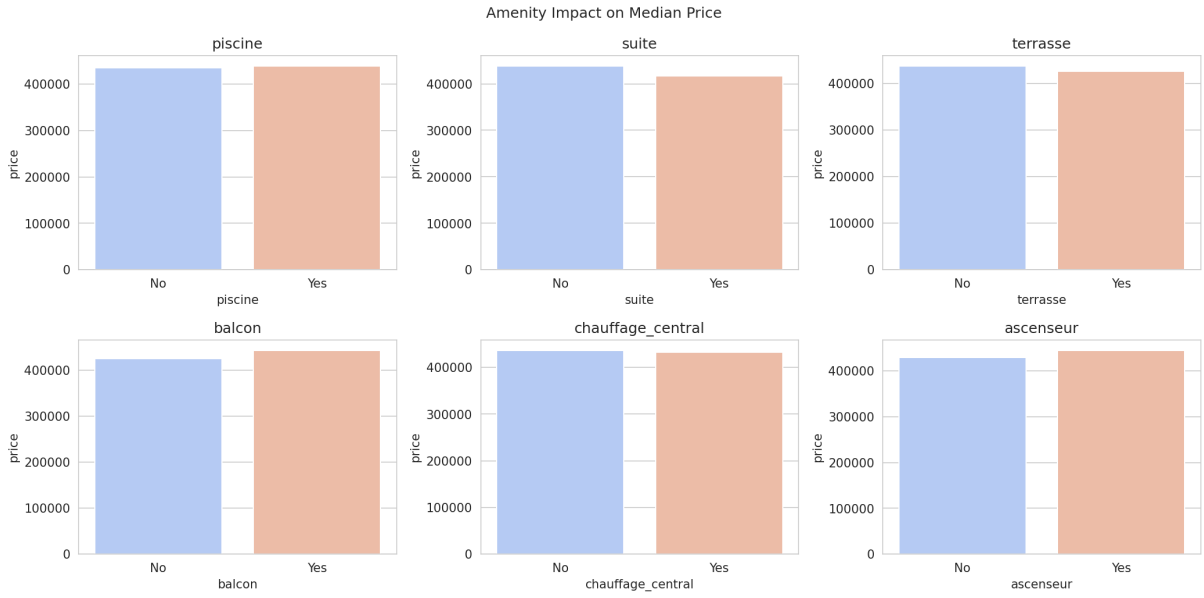


Figure 6: Median price split by amenity presence (No = 0, Yes = 1). A swimming pool (`piscine`) produces the largest premium — properties with a pool have a median price roughly twice that of properties without. A private suite (`suite`) and a terrace (`terrasse`) also carry meaningful premiums. A balcony (`balcon`) and central heating show smaller but visible impacts.

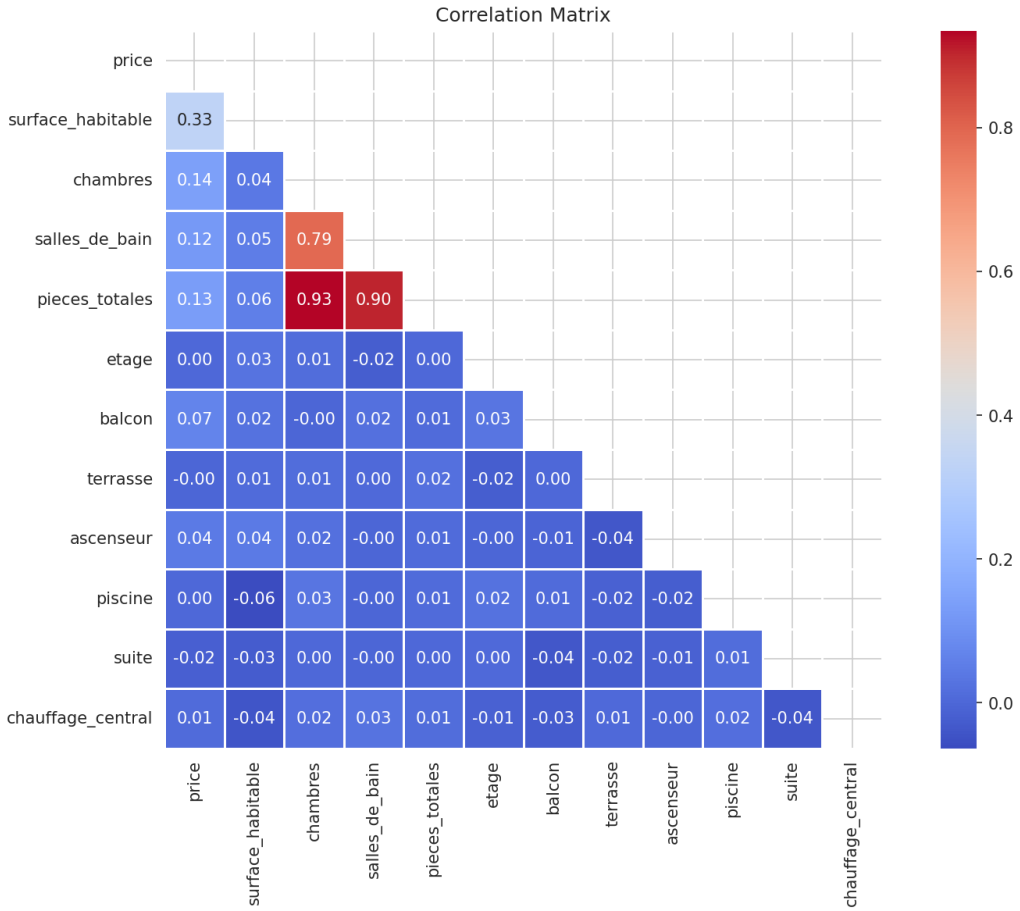


Figure 7: Lower-triangular correlation matrix of all numeric features. Key observations: (1) `surface_habitable` has the strongest linear correlation with `price` ($r = 0.34$); (2) room-count variables (`chambres`, `salles_de_bain`, `pieces_totales`) are very strongly correlated with each other ($r > 0.90$), reflecting the structural relationship that larger homes have more of every room type — these columns are nearly redundant with each other but collectively useful.

Data Preparation

The pipeline is applied in strict order to prevent **data leakage** (using test-set information to influence training). Each step is described below.

Step 1. Missing value imputation — Numeric columns filled with column median; `cuisine_type` filled with mode (`non equipee`). Medians are more robust than means because they are unaffected by outliers.

Step 2. Outlier removal — Domain-knowledge bounds remove 7 physically implausible records (0.5%): prices outside 60k–2.5M TND, surface outside 20–500 m^2, more than 8 bedrooms, or more than 15 total rooms.

Step 3. Feature engineering — Three new features capture information not directly present in any single raw column (Table 4).

Step 4. Categorical encoding — Ordinal encoding (1–3) for `cuisine_type`; one-hot encoding for `residence_type`; neighbourhood encoded as `neigh_price_index` (median price per neighbourhood).

Step 5. Train / Val / Test split — 70 / 15 / 15 with stratification on price tier to ensure all tiers are represented in every split.

Step 6. Feature scaling — `StandardScaler` fitted on the training set *only*. Validation and test sets are transformed with the training-set parameters to prevent leakage.

Table 4: Engineered Features

Feature	Formula	Rationale
`space_per_room`	surface / (chambres + SDB + 1)	Captures comfort level: high = spacious layout
`luxury_score`	Sum of 6 boolean amenities (0–6)	Single index aggregating all amenities
`neigh_price_index`	Median price of the neighbourhood	Encodes location prestige as a numeric value
`log_price`	$\ln(1 + \text{price})$	Regression target; reduces skewness from 1.88 to 0.15

```
df_clean = df.copy()

# Step 1     Impute medians
for col in ['chambres', 'salles_de_bain', 'etage', 'surface_habitable']:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
df_clean['cuisine_type'] = df_clean['cuisine_type'].fillna('non equipee')

# Step 2     Outlier removal
df_clean = df_clean[
    (df_clean['price'].between(60_000, 2_500_000)) &
    (df_clean['surface_habitable'].between(20, 500)) &
    (df_clean['chambres'] <= 8) &
    (df_clean['pieces_totales'] <= 15)
].reset_index(drop=True)

# Step 3     Feature engineering
df_clean['space_per_room']   = df_clean['surface_habitable'] / \
    (df_clean['chambres'] + df_clean['salles_de_bain'] + 1)
df_clean['luxury_score']     = df_clean[
    ['balcon','terrasse','ascenseur','piscine','suite','chauffage_central']
```

```
].sum(axis=1)
neigh_median = df_clean.groupby('neighborhood')['price'].median()
df_clean['neigh_price_index'] = df_clean['neighborhood'].map(neigh_median)
df_clean['log_price']         = np.log1p(df_clean['price'])
```

Listing 1: Imputation, outlier removal, and feature engineering

Methods

Regression Task

Three models are benchmarked. All predictions are produced in log-price space and back-transformed via $\hat{p} = e^{\hat{y}} - 1$ before computing R^2 and RMSE in TND. Model selection is based on validation performance; the chosen model is evaluated *once* on the test set.

KNN Regressor ($k = 10$)

KNN predicts the price of a new property by finding the 10 most similar properties in the training set and averaging their log-prices. "Similarity" is measured by Euclidean distance in the 20-dimensional scaled feature space. The model makes no assumptions about the shape of the relationship between features and price.

Key weakness: In high-dimensional spaces, distances become increasingly uniform (the "curse of dimensionality"), weakening the notion of "similar" neighbours.

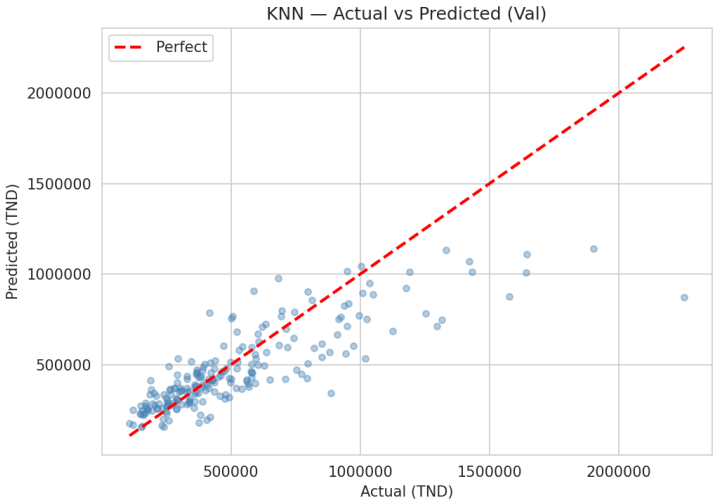


Figure 8: KNN ($k = 10$): Actual vs. Predicted prices on the validation set. The dashed red line is the perfect-prediction diagonal. Points lying far above or below this line are large errors. The spread around the diagonal is noticeable, especially at higher prices.

Decision Tree Regressor (max depth = 12)

A decision tree partitions the feature space using a hierarchy of if/then rules, each split chosen to minimise the MSE of the resulting sub-groups. The model is scale-invariant (no normalisation needed) and handles the heterogeneous mix of boolean flags, ordinal scores, continuous measurements, and one-hot dummies natively.

Key weakness: A single tree has **high variance** — a small change in the training data can reshape the entire tree, leading to unstable predictions.

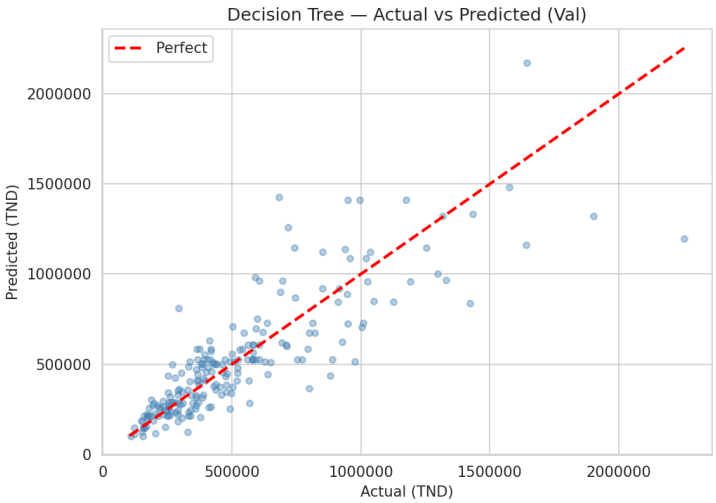


Figure 9: Decision Tree (depth = 12): Actual vs. Predicted on the validation set. Performance is visibly better than KNN, with points clustering more tightly around the perfect-prediction diagonal, particularly in the medium-price range.

Random Forest Regressor (300 trees)

A Random Forest builds 300 decision trees, each trained on a different **bootstrap sample** (random sampling with replacement) of the training data, and each split considering only a random subset of features. The final prediction is the average of all 300 tree predictions.

This **dual randomisation** has two important effects:

- It **decorrelates** the trees, so their errors do not all point in the same direction.
- Averaging over many decorrelated estimators drastically reduces the variance that limits a single Decision Tree, while preserving low bias.

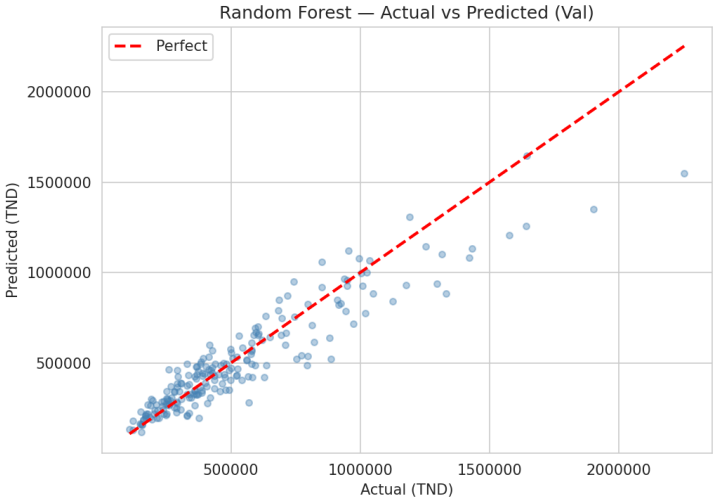


Figure 10: Random Forest (300 trees): Actual vs. Predicted on the validation set. Points cluster much more tightly around the perfect-prediction diagonal than either KNN or Decision Tree, confirming the variance-reduction benefit of the ensemble.

Classification Task

Two classifiers assign each property to one of four price tiers. All metrics are weighted averages over the four classes to account for the class imbalance in which Medium (≈49%) dominates.

Gaussian Naive Bayes (baseline)

Naive Bayes applies Bayes' theorem under the strong assumption that all features are mutually independent given the class label. This assumption is **multiply violated** in this dataset:

- Room-count variables are correlated at $r > 0.90$ (the opposite of independent).
- Engineered features are algebraic combinations of raw features.
- One-hot residence dummies are mutually exclusive by construction.

This model is included as a **diagnostic baseline** to illustrate concretely what happens when model assumptions conflict with the data.

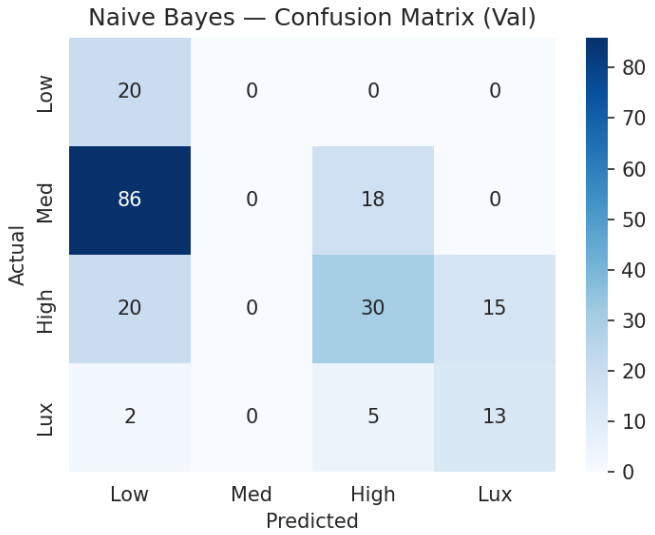


Figure 11: Naive Bayes confusion matrix on the validation set. The model classifies almost everything as "Medium" (the dominant class), producing near-zero recall for Low, High, and Luxury tiers. This is a textbook example of assumption violation causing model collapse.

Random Forest Classifier (300 trees)

The same ensemble logic as the regressor, adapted for classification: each tree votes for a class label and the majority vote wins. Key advantages for this problem:

- Bootstrap sampling naturally handles class imbalance by exposing each tree to different class proportions.
- Split-based partitioning exploits feature interactions without requiring normality assumptions.
- No feature scaling is required.

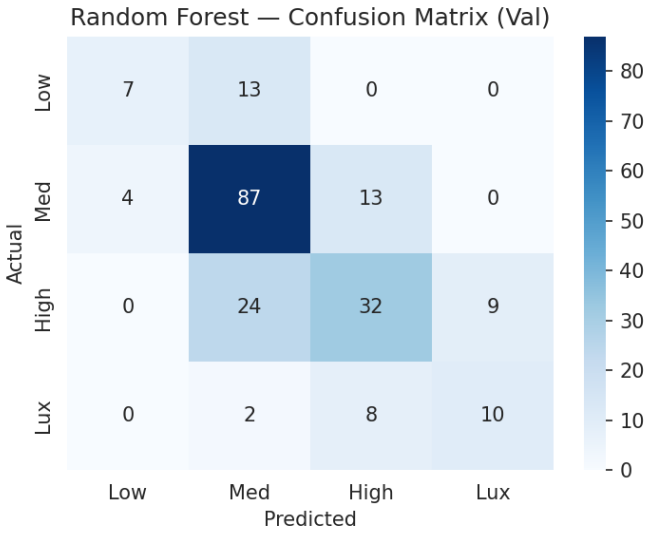


Figure 12: Random Forest classifier confusion matrix on the validation set. The diagonal (correct predictions) is now dominant for all four classes. The model correctly identifies the majority of Medium properties and captures a meaningful share of Low and High tiers.

Results

Regression

Table 5: Regression Performance — Validation Set (model selection)

Model	R^2	RMSE (TND)
KNN ($k = 10$)	0.641	≈208,000
Decision Tree	0.696	≈192,000
Random Forest	**0.852**	**≈134,000**

Random Forest dominates on both metrics, explaining 85.2% of price variance on the validation set with a mean error of ≈134,000 TND. It is selected as the best regressor. The model is then evaluated *once* on the untouched test set.

Table 6: Random Forest Regressor — Validation vs. Test

Metric	Validation	Test
R^2	0.852	0.747
RMSE (TND)	≈134,000	≈155,000

Interpreting the Val → Test Gap

The drop from $R^2 = 0.852$ (validation) to 0.747 (test) is **normal generalisation loss**, not overfitting. Validation is used for model selection, which introduces a mild optimism bias. The test set is truly unseen. A 10-point R^2 gap is typical for ensemble models on small datasets ($n \approx 1{,}400$).

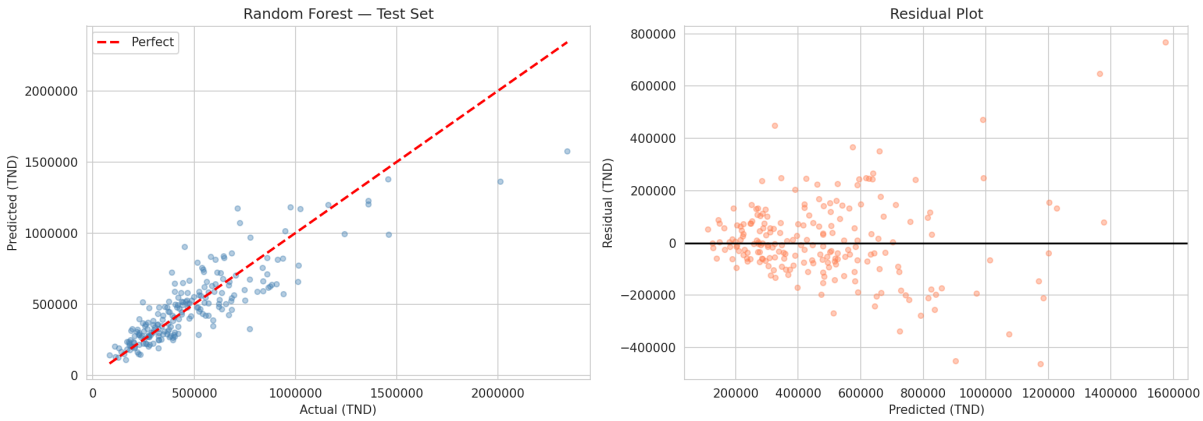


Figure 13: **Left:** Actual vs. Predicted on the test set. The strong alignment with the perfect-prediction diagonal confirms that the Random Forest generalises well to unseen data. **Right:** Residual plot (Predicted vs. Residual). Residuals are scattered symmetrically around zero with no obvious fan-shape or trend, indicating that model assumptions hold and there is no systematic bias.

Table 7: Top 5 Features — Random Forest Regressor

Rank	Feature	Importance
1	`neigh_price_index`	32.7%
2	`type_villa`	22.4%
3	`type_penthouse`	12.0%
4	`surface_habitable`	9.7%
5	`type_studio`	6.0%

Feature Importances

These importances align with the EDA findings: neighbourhood prestige drives more than twice the variance explained by living area, and property type (villa + penthouse combined: 34.4%) is a stronger signal than any single physical measurement.

Classification

Table 8: Classification Performance — Validation Set (model selection)

Model	Accuracy	Precision	Recall	F1
Naive Bayes	0.301	0.235	0.301	0.236
Random Forest	**0.651**	**0.643**	**0.651**	**0.637**

Reading the Results

The Random Forest achieves **74.6% accuracy** and a **weighted F1 of 0.745** on the test set, well above the 49% majority-class baseline. Medium has the highest F1 (0.81) because it is both the largest and most internally homogeneous class. Luxury has the lowest recall (0.57), partly because only 21 test examples are available to evaluate it on, making this estimate less reliable.

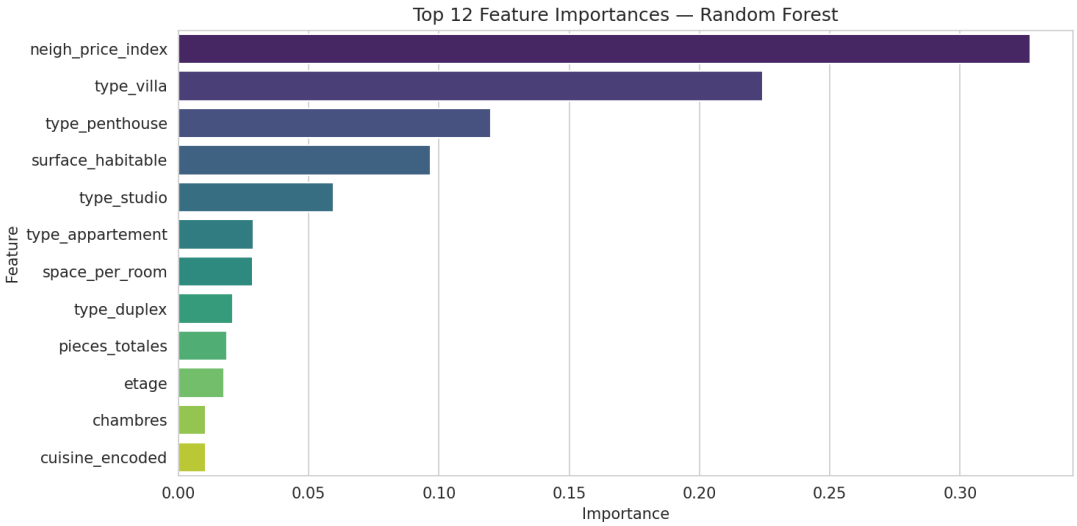


Figure 14: Top-12 feature importances from the Random Forest Regressor (mean decrease in impurity). `neigh_price_index` is by far the dominant driver (32.7%), followed by the villa and penthouse type dummies. Living area (`surface_habitable`) ranks fourth. Boolean amenity features and floor number contribute much less individually.

Table 9: Detailed Classification Report — Random Forest (Test Set)

Class	Precision	Recall	F1	Support
Low (<200k)	0.81	0.65	0.72	20
Medium (200–500k)	0.79	0.83	0.81	103
High (500k–1M)	0.66	0.71	0.68	65
Luxury (>1M)	0.80	0.57	0.67	21
Weighted avg	0.75	0.75	0.75	209

Naive Bayes Failure — Pedagogical Note

Naive Bayes achieves F1 = 0.24 — barely above random guessing (25% for four classes). This is a direct consequence of the independence assumption being violated by correlated room-count features, engineered combinations, and mutually exclusive dummies. It illustrates a general principle: a well-known model applied to an incompatible data structure produces results no better than a simplistic heuristic.

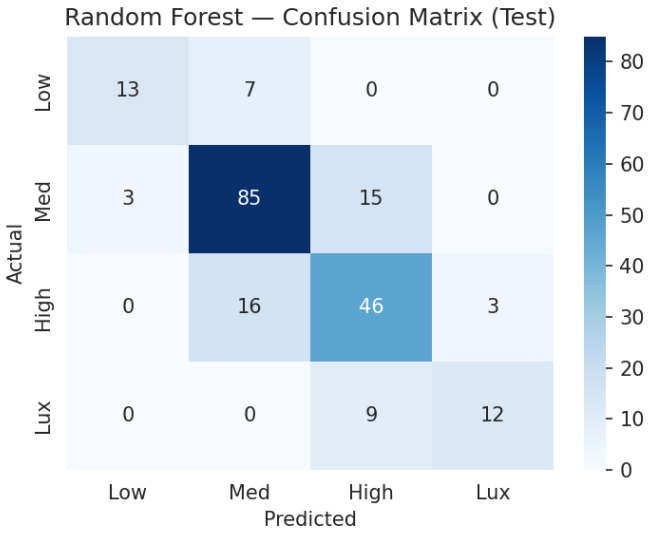


Figure 15: Random Forest confusion matrix on the test set. The model performs best on the Medium class (largest support) and shows competitive performance across all tiers. Most errors occur at adjacent tier boundaries (e.g. High misclassified as Medium), which is physically reasonable since the 500k TND cut-off is arbitrary.

Live Prediction Demo

Both the best regressor and the best classifier can be applied to any new property in under a second. The example below uses a 3-bedroom, 120 m² apartment in *El Manar 2* with a balcony and elevator, on floor 2, with an equipped kitchen.

```
sample = {
    'surface_habitable': 120.0, 'chambres': 3.0,
    'salles_de_bain': 2.0, 'pieces_totales': 6.0, 'etage': 2.0,
    'balcon': 1, 'terrasse': 0, 'ascenseur': 1,
    'piscine': 0, 'suite': 0, 'chauffage_central': 0,
    'neighborhood': 'El Manar 2',
    'residence_type': 'appartement',
    'cuisine_type': 'equipee',
}
# Compute engineered features, build demo_df aligned to X.columns
predicted_log   = rf_reg.predict(demo_df)[0]
predicted_price = np.expm1(predicted_log)
predicted_tier  = rf_clf.predict(demo_df)[0]
tier_map = {0:'Low', 1:'Medium', 2:'High', 3:'Luxury'}
print(f"Estimated price : {predicted_price:,.0f} TND")  # -> 299,470 TND
print(f"Predicted tier  : {tier_map[predicted_tier]}")  # -> Medium
```

Listing 2: Live property pricing demo

Demo Output

Property:	3-bed apartment · 120 m² · El Manar 2 · Floor 2 · Equipped kitchen
Estimated price:	**299,470 TND**
Predicted tier:	**Medium (200k–500k TND)**

Discussion and Limitations

Key Findings

- **Log-transformation was essential.** Without it, every regression model suffers substantially higher error because the right-skewed distribution pulls residuals upward for mid-range properties.
- **Random Forest beats both competitors** because its ensemble mechanism directly addresses the weaknesses of each: distance degradation in high dimensions (KNN) and variance instability (single Decision Tree).
- **Location is the dominant price driver.** The engineered `neigh_price_index` accounts for 32.7% of Random Forest importance — more than living area, bedroom count, and all amenities combined.
- **Property type is the second strongest signal.** Villa and penthouse dummies together account for 34.4% of importance, confirming that "what you build" matters nearly as much as "where you build it."
- **Naive Bayes is a cautionary tale.** Its F1 of 0.24 demonstrates empirically what model theory predicts: the independence assumption is so severely violated that the classifier cannot discriminate between tiers.

Limitations

1. **Listing prices, not transaction prices.** Actual sale prices may differ significantly from asking prices in a negotiation-heavy market. The model reflects seller expectations, not market-clearing values.
2. **No temporal dimension.** The dataset is a static snapshot. The model cannot capture price trends driven by inflation, interest rate changes, or macroeconomic shifts.
3. **Property condition not captured.** Two identical-attribute listings may differ by hundreds of thousands of TND based on renovation quality or interior finishes, which are invisible to the model.
4. **Target-encoding leakage.** `neigh_price_index` was computed on the full cleaned dataset before splitting. In production, it should be derived from historical data external to the training set to avoid information leakage.
5. **Small dataset.** With 1,393 usable records and 4 classes including only 133 Low and 137 Luxury examples, metrics for minority tiers should be interpreted cautiously.

Conclusion

This project applied data mining techniques to 1,393 Tunisian real estate listings to build and evaluate price prediction models. The data preparation pipeline included median imputation, domain-knowledge outlier removal, three engineered features, ordinal and one-hot encoding, a stratified 70/15/15 split, and leakage-free `StandardScaler` normalisation.

Three regression models and two classifiers were benchmarked on a held-out validation set, with the best model evaluated once on the test set:

- **Best regressor:** Random Forest — $R^2 = 0.747$, RMSE ≈155,000 TND on the test set. The residual plot shows no systematic bias.

- **Best classifier:** Random Forest — Accuracy = 74.6%, weighted F1 = 74.5%, compared to a 49% majority-class baseline.
- **Top price drivers:** neighbourhood prestige (32.7%), property type – villa + penthouse combined (34.4%), and living area (9.7%).
- **Naive Bayes failure** (F1 = 0.24) illustrates the cost of applying a model whose independence assumptions are multiply violated by the data structure.

Future work should incorporate transaction prices (rather than listing prices), add a temporal dimension to capture market trends, and use out-of-fold target encoding for `neigh_price_index` to eliminate the leakage identified above.

Data Mining Project — Spring Semester 2025–2026
Noursine Amira · Lina Brahmi · Maycem Azaza
University of Tunis, Tunis Business School — Bachelor in Business Analytics